The CourseDesk incident handover

Bash in GitHub Codespaces · about 100 minutes · Name: _____ Date: _____

Your job

You are a junior developer on the Platform day team at Larkspur College. Overnight, **coursedesk** (the course registration portal) returned errors during the late-registration window. Sam Okafor, the night on-call engineer, left a draft handover note before going home. The draft uses an old format, guesses at the numbers and says the cause is unknown.

By the end of this lab you will hand the day team a **handover package** in `work/`:

File	What it gives the next engineer
`work/handover.txt`	A corrected 10-line handover note in the current runbook format
`work/web-02.conf`	The faulty server's configuration, reconciled with the approved version
`work/students-to-contact.txt`	Students whose enrolment failed, for Student Services
supporting files	Counts and comparisons that prove each number in the note

Each experiment teaches one tool and produces a fact or file that the final task uses. Write each fact in `work/findings.txt` (you create it in Experiment 2) as soon as you find it.

Set up

1. Open `https://codespaces.new/dhruvgupta-24/software-engineering-terminal-lab` (or, on the repository page, choose **Code** → **Codespaces** → **Create codespace on main**). Create the Codespace and wait until the terminal prints `Setup complete`. If a terminal was already open before that message, open a new one (**Terminal** → **New Terminal**) so the lab settings apply.
2. In the terminal, at the repository root, create your attempt and move into it:

```
bash lab/start.sh
cd attempts/attempt-01
pwd
ls evidence work
```

`start.sh` prints the exact `cd` command; use the folder name it prints. The folder you are now in is your **lab root**. Write its full path (from `pwd`): _____

Folder	Contents	Rule
`evidence/`	Original files: `runbook.txt`, `requests.log`, `alerts.log`, `web-02-live.conf`, `web-02-approved.conf`, `handover-draft.txt`	Read only. The files are write-protected so a mistake cannot destroy them.
`work/`	Your working copies (`handover.txt`, `web-02.conf`) and every file you create	Edit and save here only.

Starting again. From the repository root, run `bash lab/start.sh` again. It creates the next folder (`attempt-02`, and so on) with fresh copies. Earlier attempts are never changed or deleted.

How to work. Start every experiment in your lab root. Type each command yourself, one at a time, and read the output before moving on. Where the book asks for a prediction, write it **before** you run the command. Click inside the terminal before using a Ctrl shortcut (on a Mac, use Control, not Command).

Experiment 1 · Read the runbook with `less`

Goal: find the current handover format, the current incident lead and the log field numbers. · **Start in:** lab root · **Produces:** facts for `work/findings.txt`

`evidence/runbook.txt` is 264 lines long. `cat` would scroll most of it off the screen. **`less`** shows a long file one screen at a time, lets you move backwards and search, and never changes the file. While `less` is open, your keys go to `less`, not to Bash.

Key in `less`	Action
`Space` / `b`	Forward one screen / back one screen
`g` / `G`	Jump to the first line / the last line (`G` is Shift+g)
`/` text `Enter`	Search forward for text. Searches are case-sensitive.
`n` / `N`	Next match / previous match of the last search
`q`	Quit and return to the Bash prompt

1A. Move around (guided). Run `less evidence/runbook.txt`. Press `Space` twice, then `b` once. Press `G`: the last line should read `END OF RUNBOOK`. Press `g` to return to the title. The bottom line of the screen shows the file name or a `:` prompt. That means you are still inside `less`.

1B. Search and step between matches (guided). The runbook keeps retired sections for audit history, so the first match is not always the one to follow.

Prediction: how many headings contain `NOTE FORMAT`? _____

Type `/NOTE FORMAT` and press `Enter`. You land on Section 9. Read its heading. Press `n` to reach the next match, then `N` to go back once, then `n` again. Stay on the section marked **CURRENT** and read its ten lines.

Which revision of the handover format is current? _____ Why must you reject the first match? _____

1C. Find two more facts (independent). Still inside `less`. A search runs forward from the current screen, so press `g` before each new search.

- Search for `ROTA`. Use `n` until you reach the CURRENT rota. Current incident lead: _____
- Search for `LOG FORMAT`. Field numbers in `requests.log`: status ____ response time ____ endpoint ____ student ID ____

Press `q` to quit. If you see `:` or `(END)` at the bottom of the screen, you are still in `less`. Press `q` before typing a shell command.

Experiment 2 · Edit and create files with `nano`

Goal: start correcting the handover note and create your findings file. · **Start in:** lab root · **Produces:** `work/handover.txt` (lines 1 to 3), `work/findings.txt`

`nano` is a text editor that runs inside the terminal. `nano FILE` opens FILE if it exists, or starts a new, empty file with that name if it does not. The bottom two lines of the screen list shortcuts: `^O` means Ctrl+O. Do not type the `^` character.

Key in `nano`	Action
Arrow keys, `Backspace`	Move the cursor, delete the character before it
`Ctrl+O`, then `Enter`	Write (save). nano shows `File Name to Write:` with the current name; `Enter` accepts it.
`Ctrl+X`	Exit. If there are unsaved changes, nano asks `Save modified buffer?`: press `Y`, then `Enter`.
`Ctrl+K`	Cut (delete) the whole line under the cursor

2A. Edit an existing file (guided). Run `cat work/handover.txt` to see the draft, then `nano work/handover.txt`. Rev 4 begins with these three lines:

```
Incident: INC-2417
Service: coursedesk
Escalation: <current incident lead>
```

Line 1 is already correct. Change line 2 from `Owner: Sam Okafor` to `Service: coursedesk`. On line 3, replace `Ravi Menon` (from the archived rota) with the current incident lead you found in 1C. Leave lines 4 to 6 for the final task. Save with `Ctrl+O`, `Enter`, and exit with `Ctrl+X`.

Verify: run `cat work/handover.txt`. Then open the file in nano again and exit with `Ctrl+X` without changing anything. Observation: did nano ask to save this time? _____ What does that tell you about the file on disk?

_____

2B. Create a new file (independent). Run `nano work/findings.txt`. nano shows `[ New File ]`. Type these two lines, using your own answer for the name. You do not need `Enter` after the second line: nano ends the file with a newline when it saves.

```
Handover format: rev 4
Incident lead: <name from 1C>
```

Save, exit, and confirm with `cat work/findings.txt`. Add one line to this file (reopen it with nano) whenever this book says **Record**.

Check this experiment: `python3 ../../lab/check.py 2`

Experiment 3 · Select fields with `cut`

Goal: extract the status codes and find when the first 503 happened. · **Start in:** lab root · **Produces:** `work/statuses.txt`, `work/time-server-status.txt`

Each line of `evidence/requests.log` is one request with six fields separated by `|`, as you found in the runbook:

```
01:02:27|web-02|200|29|/health|-
```

`cut` prints selected fields from every line and keeps the lines in their original order.

Option	Meaning
`-d "|"`	The delimiter: the character between fields. Quote it. Unquoted, Bash reads `|` as a pipe between two commands.
`-f 3` / `-f 1,2,3`	The field number(s) to keep, counting from 1. Several fields are printed with the delimiter between them.

3A. One field (guided). Predict first: what will Bash do with `cut -d | -f 3 evidence/requests.log`? _____ Run it and read both error messages. Bash tried to run `cut -d` and a command called `-f`. Now run the correct form and page through it with `less`:

```
cut -d "|" -f 3 evidence/requests.log | less
```

Press `q`, then save the same output with `>`:

```
cut -d "|" -f 3 evidence/requests.log > work/statuses.txt
```

Line 1 of `work/statuses.txt` is the status of the first request in the log. This matters later: because `cut` keeps row order, line N of the output always belongs to line N of the log.

3B. Several fields (independent). Write and run a command that saves fields 1, 2 and 3 (time, server, status) of every request to `work/time-server-status.txt`. Your command: _____

Then list only the 503 responses:

```
grep -F "|503" work/time-server-status.txt | less
```

`"|503"` includes the bar so the search matches the status field, not a time or another number containing 503. Because row order was kept, the first line is the earliest 503. **Record** `First 503: <time> on <server>` in `work/findings.txt`.

Observation: run `cut -d "|" -f 5,4 evidence/requests.log | less`. Which field comes first in the output? _____ `cut` always prints fields in their original order, whatever order you list them in.

Check: `python3 ../../lab/check.py 3`

Experiment 4 · Order lines with `sort`

Goal: find the slowest request. · **Start in:** lab root · **Produces:** `work/latency-ranked.txt`

`sort` prints the lines of its input in order. It does not change the input file. By default it compares lines as **text**, character by character, from the left.

Option	Meaning
(none)	Text order: `10012` comes before `12`, because the second characters `0` and `2` are compared
`-n`	Numeric order: read the number at the start of each line and compare values
`-r`	Reverse the order (largest or last first). Combine as `-n -r` for highest number first.

4A. Text versus numbers (guided). Response time is field 4. Predict: in text order, will the first line be the smallest time? _____

```
cut -d "|" -f 4 evidence/requests.log | sort | less
```

Press `g` and read the first lines, then `G` for the last. Quit with `q`. Now run the same pipeline with `sort -n` and look again. Where does `10093` appear with each command? Text: _____ Numeric: _____

4B. Rank and save (guided). Keep the endpoint next to each time by cutting fields 4 and 5. `sort -n` uses the number at the start of each line (`10093|/api/enrol` sorts by 10093):

```
cut -d "|" -f 4,5 evidence/requests.log | sort -n -r > work/latency-ranked.txt
less work/latency-ranked.txt
```

The first line is the slowest request. **Record** `Slowest request: <ms> ms on <endpoint>`.

4C. Never redirect onto the input (guided). Copy a file you can afford to lose, predict, then run:

```
cp work/statuses.txt work/scratch.txt
sort work/scratch.txt > work/scratch.txt
cat work/scratch.txt
```

Prediction: what will `cat` show? _____ Observed: _____. The shell prepares `> work/scratch.txt` **before** `sort` starts: it empties the destination file, so `sort` then reads an empty file. Always write to a different file from the one you read. (Trying this on a file in `evidence/` fails with `Permission denied`, which is why the evidence is write-protected.) Delete the scratch file with `rm work/scratch.txt`.

Check: `python3 ../../lab/check.py 4`

Experiment 5 · Group repeated lines with `uniq`

Goal: summarise the pager alerts and count every status code. · **Start in:** lab root · **Produces:** `work/alert-types.txt`, `work/alert-counts.txt`, `work/status-counts.txt`

`uniq` removes a line when it is identical to the line **directly above it**. It compares only neighbouring lines. `uniq -c` prints each group once with the number of lines in the group in front. The pager repeats an alert every minute, so `evidence/alerts.log` (time | alert | server) has many repeats.

5A. Adjacent repeats (guided). Look at the alert and server fields, then predict how many lines `uniq` will print:

```
cut -d "|" -f 2,3 evidence/alerts.log
```

Prediction: _____ lines.

```
cut -d "|" -f 2,3 evidence/alerts.log | uniq
```

Observed: _____ lines. Name one pair that still appears more than once: _____ It survived because its repeats were not next to each other.

5B. Sort first (guided). `sort` puts identical lines next to each other, so `uniq` can then keep exactly one of each:

```
cut -d "|" -f 2,3 evidence/alerts.log | sort | uniq > work/alert-types.txt
cat work/alert-types.txt
```

5C. Count, and see why order matters (guided). Run both pipelines and compare:

```
cut -d "|" -f 2,3 evidence/alerts.log | uniq -c
cut -d "|" -f 2,3 evidence/alerts.log | sort | uniq -c > work/alert-counts.txt
cat work/alert-counts.txt
```

The same three commands in a different order give a different answer. Without `sort`, `HighErrorRate|web-02` is split into several small groups and none of them shows the real total. Leading spaces before each count are normal. To rank alerts by frequency, sort the counts numerically: `sort -n -r work/alert-counts.txt`.

5D. Count every status (independent). Using `work/statuses.txt` from Experiment 3, save a count of each status code to `work/status-counts.txt`. Your command: _____ **Record** `503 responses: <count>`.

Check: `python3 ../../lab/check.py 5`

Experiment 6 · Find and reuse commands with `history`

Goal: reuse your 503 search from Experiment 3 to count 503s per server. · **Start in:** lab root · **Produces:** `work/errors-by-server.txt`

Bash remembers the commands you type. **`history`** lists them with numbers; `history 10` shows the last ten. Your numbers will differ from a classmate's. **Reverse search** (Ctrl+R) finds an earlier command by typing part of it.

Keys at the Bash prompt	Action
`Ctrl+R`, then text	Show the most recent command containing that text
`Ctrl+R` again	Show the next older match
`Right Arrow`	Accept the match onto the prompt for editing, **without running it**
`Enter`	Run the match (or the edited line)
`Ctrl+G`	Cancel the search and return to an empty prompt; nothing runs

6A. List and filter (guided). Run `history 10`. Then filter the whole list with a pipe:

```
history | grep -F "uniq"
```

The last line is usually this `history | grep` command itself, because it also contains `uniq`. Number of your `sort | uniq -c` command for alerts: _____

6B. Search and cancel (guided). At an empty prompt press `Ctrl+R` and type `uniq -c`. The prompt shows your most recent match. Press `Ctrl+R` again to step to an older one. Press `Ctrl+G`. The prompt is empty again and nothing ran.

6C. Edit an old command (independent). Press `Ctrl+R` and type `|503`. Stop when the prompt shows your command from 3B, `grep -F "|503" work/time-server-status.txt | less`. Press `Right Arrow`, then `End` (or `Ctrl+E`) to move the cursor to the end of the line. Now edit the line: remove `| less` and add a pipeline that keeps only the server field, counts each server and ranks the counts highest first, saving to `work/errors-by-server.txt`. Read the whole line before pressing `Enter`.

My edited command: _____

Run `cat work/errors-by-server.txt`. **Record** `Faulty server: <server with the most 503 responses>`.

If `Ctrl+R` does nothing, click inside the terminal and try again. Use `history | grep` as a fallback and tell your instructor.

Check: `python3 ../../lab/check.py 6`

Experiment 7 · Compare and repair with `diff`

Goal: find what deploy D-118 changed on web-02, then reconcile your working copy. · **Start in:** lab root · **Produces:** `work/config-changes.diff`, repaired `work/web-02.conf`

`diff FIRST SECOND` compares two files line by line and prints instructions that would turn FIRST into SECOND. **No output means the files are identical.** Output is not an error: when files differ, the differences are the answer you asked for.

7A. Identical files (guided). Your working copy started as a copy of the live file:

```
diff evidence/web-02-live.conf work/web-02.conf
```

Observed: _____

7B. Changed files (guided). Compare the live file (first) with the approved file (second):

```
diff evidence/web-02-live.conf evidence/web-02-approved.conf
```

Marker	Meaning
`7c7`	Line 7 of FIRST must **change** to become line 7 of SECOND
`9d8`	Line 9 of FIRST must be **deleted**; SECOND continues after its line 8
`10a10`	After line 10 of FIRST, **add** line 10 of SECOND
`<` / `>`	A line from FIRST (live) / a line from SECOND (approved)
`---`	Separates the old and new lines of one change

Which setting was changed, and from what to what? _____ Which line was added by the deploy? _____ Which approved line is missing from the live file? _____

Prediction: if you swap the two file names, what happens to `<`, `>`, `a` and `d`? _____ Try it and check.

Save the comparison in its normal order. diff reports that differences exist, which is expected here; the file is still saved:

```
diff evidence/web-02-live.conf evidence/web-02-approved.conf > work/config-changes.diff
```

7C. Repair and verify (independent). Open `work/web-02.conf` in nano and make the three changes the comparison describes (`Ctrl+K` deletes a whole line). Save, exit, then verify:

```
diff work/web-02.conf evidence/web-02-approved.conf
```

Repeat until this prints nothing. **Record** `Cause: <setting> changed from <approved value> to <live value>`.

Check: `python3 ../../lab/check.py 7`

Final task · Complete the handover package

Start in: lab root · **Uses:** every tool and your `work/findings.txt` · **Produces:** `work/students-to-contact.txt`, final `work/handover.txt`

This task gives requirements, not commands. Use intermediate files in `work/` if a long pipeline is hard to follow. Several clear commands are as good as one long one.

1. **Read the rules.** In `less`, read runbook Section 12 (student communications) and Section 11 (why a 504 is treated differently).
2. **Students to contact.** Create `work/students-to-contact.txt` exactly as Section 12 requires. Before you start, write the order in which you will use `grep`, `cut`, `sort` and `uniq`, and why `sort` must come before `uniq`:

   _____
3. **Handover note.** Edit `work/handover.txt` in nano so it contains exactly the ten lines of the rev 4 format, in order, with every value from your findings. Delete the old rev 3 lines.
4. **Verify.** Compare `work/web-02.conf` with the approved file once more, view your note with `cat`, and run the full checker.

Number of students to contact: _____ Total 503 responses: _____ Faulty server: _____

Validation

From your lab root, run:

```
python3 ../../lab/check.py          # every check
python3 ../../lab/check.py 5        # one experiment: 2 to 7, or final
```

The checker reads your `work/` files and confirms that every file in `evidence/` still matches the original. It prints `PASS` or `FAIL` for each check with a reason, and ends with a score. It **never changes your files**, so you can run it as often as you like. It cannot see how you used `history`, `nano` or `less`; those skills are signed off below.

See a failing check, then fix it (do this once). Make a deliberate mistake, check it, repair it, and check again:

```
cut -d "|" -f 2,3 evidence/alerts.log | uniq > work/alert-types.txt
python3 ../../lab/check.py 5
```

Expected: `FAIL ... still has repeated lines. uniq removes only adjacent repeats; sort first, then uniq.` Recall your correct 5B command with `Ctrl+R`, run it, and run the check again. Expected: `PASS`. Message I saw:

_____

When is diff output expected? In 7B and in `work/config-changes.diff`, differences are the result you wanted: they show what the deploy changed. In 7A and after your repair in 7C, the correct result is **no output**, which means the files are identical. A message such as `No such file or directory` is different: it means a file name is wrong.

Skills sign-off. Your instructor watches you do each row, on the files named or on a variation they choose.

Tool	Show this	Instructor
`history`	`history 10`; `history | grep -F` a word; `Ctrl+R` to a match, `Ctrl+R` to an older match; `Right Arrow`, edit and explain the command before running it; `Ctrl+G` to cancel a search	_____
`nano`	Open `work/findings.txt`, add a line, `Ctrl+O` + `Enter`, `Ctrl+X`; reopen to show it was saved; create a new file	_____
`less`	In `evidence/runbook.txt`: `Space`, `b`, `G`, `g`; `/` search, `n` and `N` to the CURRENT section; `q`	_____

Checker score: _____ / 14

Command reference

Command or keys	Purpose
`less FILE` · `Space` `b` `g` `G` `/text` `n` `N` `q`	Read a long file: page, jump, search, quit
`nano FILE` · `Ctrl+O` `Enter` · `Ctrl+X` · `Ctrl+K`	Edit or create a file: save, exit, cut a line
`cut -d "|" -f 2` · `cut -d "|" -f 1,3`	Keep field 2, or fields 1 and 3, of every line, in row order
`sort` · `sort -n` · `sort -n -r`	Text order · numeric order · highest number first
`sort FILE | uniq` · `sort FILE | uniq -c`	One line per distinct value · count each value
`diff FIRST SECOND` · `diff A B > FILE`	Compare (`<` FIRST, `>` SECOND, `a` `c` `d`) · save the comparison
`history 10` · `history | grep -F "text"`	Recent commands · search the list
`Ctrl+R` · `Ctrl+R` · `Right Arrow` · `Enter` · `Ctrl+G`	Search history · older match · edit · run · cancel
`A | B` · `A > FILE`	Send A's output to B · save output (replaces FILE; never the input file)